14. Pre-Trained Neural Network Models

Why reinvent the wheel when millions of images have already done the training for you?

Previous Section:

13. Convolutional Neural Networks

Contents:

Before we start building image classifiers with powerful neural networks, it is worth asking a simple question: *Why do researchers keep using pre-trained models instead of training everything from scratch?*

After all, if Machine Learning is about learning from data, wouldn't it make sense to simply gather our dataset and train a neural network ourselves? In theory, yes. In practice, things become much more complicated.

Training a deep neural network from scratch is expensive. Not just in terms of hardware, but also in terms of time, data, and expertise. Modern convolutional neural networks often contain millions of parameters. Teaching such a network to recognize useful visual patterns requires enormous datasets and countless training iterations.

For many organizations, this is simply impractical. And that is exactly why pre-trained models exist. A pre-trained model is a neural network that has already gone through the difficult learning process on a massive dataset. Instead of starting from zero, we begin with a model that has already spent days, weeks, or even months learning useful representations from millions of examples.

One of the most famous examples is **ImageNet**, a dataset containing approximately 1.4 million images distributed across 1,000 different categories.

Imagine spending years teaching a student to recognize animals, vehicles, buildings, plants, and everyday objects. Once that student has acquired that knowledge, it would be wasteful to erase everything and start over whenever a new task appears. Instead, we leverage what has already been learned.

The same philosophy applies to deep learning. If the original dataset is large and diverse enough, the visual features learned by the model become surprisingly general. Edges, corners, textures, shapes, and object structures learned from one problem can often be reused for many others.

The above idea is known as **Transfer Learning**. Rather than building a new neural network from scratch, we transfer the knowledge acquired from a previous task and adapt it to a new one.

For this section, we will use **VGG16**, a classic convolutional neural network trained on ImageNet. There are certainly newer and more powerful architectures available today, but VGG16 has one major advantage: ***Its architecture is straightforward.***

If you have already studied traditional Convolutional Neural Networks, VGG16 feels like a natural extension of what you already know. Its layers are organized

in a clean and understandable way, making it an excellent model for learning transfer learning concepts.

Once we obtain a pre-trained model, there are two common ways to use it.

- The first approach is **feature extraction**. We freeze the convolutional base and use its learned features while training only a new classifier for our specific task.
- The second approach is **fine-tuning**. Fine-tuning takes things one step further. After training the new classifier, we carefully unfreeze a small number of upper layers and continue training the entire network together.

The reason we only modify the upper layers is that different layers learn different levels of information:

- Early layers learn simple visual patterns such as edges and textures.
- Middle layers learn shapes and object parts.
- Later layers learn more abstract and task-specific concepts.

Since the higher layers contain more specialized knowledge, they are the best candidates for adaptation when solving a new problem.

However, fine-tuning must be done carefully. A large learning rate can easily destroy the valuable representations that the network spent millions of images learning. Therefore, fine-tuning typically uses a very small learning rate to make gradual adjustments rather than drastic changes.

In this section, our goal is not to explore the mathematical details behind VGG16 or the history of ImageNet. The researchers who created these models have already written entire papers explaining those topics far better than I can.

Our goal is much simpler: ***To learn how to use pre-trained models effectively, understand the ideas of transfer learning and fine-tuning, and see how a neural network trained on millions of images can help us solve our own problems with only a fraction of the effort.***

1. The VGG16 Model

This model exists because of a simple problem. Reading an image is useful, but it is rarely enough. Imagine showing a computer an architectural blueprint. If the model simply says: ***"This is a building blueprint".*** Then what? That answer does not tell us where the restroom is. It does not tell us where the emergency

exits are located. It does not identify the electrical system, the water pipes, or the structural supports.

The same problem appears in medical imaging. Suppose we show a model an image of the human body. If it only responds: **"This is a human body".** That is technically correct, but practically useless. We often need much more detailed understanding: ***Where is the heart? Which part is the liver? Is there any abnormality? Which blood vessel is blocked?***

To achieve this level of understanding, a neural network must learn features at different levels of complexity. Unfortunately, we cannot simply tell the model: "This shape is a heart"; "This region is a restroom"; "This line represents a wall". The model must learn these concepts from labeled examples.

This challenge inspired one of the most influential architectures in computer vision: **VGG16**. Rather than building a giant network with completely different layers everywhere, VGG16 follows several important design principles:

- **Modularity** — Complex systems are divided into smaller blocks, similar to how we organize large Python programs into classes and functions.
- **Hierarchy** — Features are learned progressively, from simple patterns to abstract concepts.
- **Reuse** — The same convolutional operations are repeatedly used throughout the network.
- **Repeated Blocks** — Instead of designing every layer differently, VGG16 relies on repeated convolutional blocks.
- **Ablation-Friendly Design** — Researchers can remove individual components and measure how much each contributes to performance.

The result is a network that is surprisingly simple to understand despite containing millions of parameters.

Below is the architecture of VGG16.

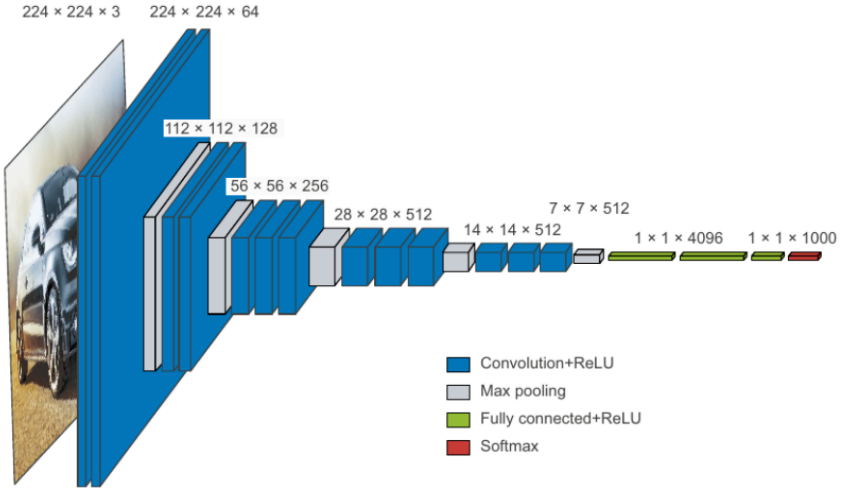


Figure 9.8 The VGG16 architecture: note the repeated layer blocks and the pyramid-like structure of the feature maps

At first glance, it looks more like a building blueprint than a neural network. Fortunately, we do not need to understand every detail immediately. Instead, we will focus on a few important patterns.

- **The Blue Blocks: Convolution + ReLU:** The blue blocks represent convolutional layers followed by ReLU activation functions.
    - If you have already studied Convolutional Neural Networks, you can think of these blocks as the "feature detectors" of the network.
    - They examine the feature maps produced by previous layers and attempt to discover increasingly complex patterns.
    - At the beginning of the network, the layers learn simple features such as: Edges; Corners; Curves; Color transitions
    - As we move deeper into the network, these features gradually combine into more meaningful structures: Wheels; Windows; Eyes; Faces; Animal bodies; Vehicle shapes. This is the hierarchical learning process that makes CNNs so powerful.

Notice how the dimensions change throughout the network:

`(224 × 224 × 3) → (224 × 224 × 64) → (112 × 112 × 128) → (56 × 56 × 256) → (28 × 28 × 512) → (14 × 14 × 512) → (7 × 7 × 512)`

For example: **224 × 224 × 64** means: Width = 224 pixels; Height = 224 pixels; Depth = 64 feature maps. The interesting part is the **64**. This number tells us that the network has produced **64 different feature maps**, each generated by a different filter. Which naturally leads to the next question.

- **What Exactly Is a Filter?** A filter (also called a kernel) is a small matrix of learnable weights that slides across an image. Think of a filter as a specialized detector. During training, each filter learns to respond strongly to a particular visual pattern.

For example: One filter may become an edge detector. Another may detect horizontal lines. Another may respond to circles. Another may specialize in textures. Another may activate when it sees eyes.

The remarkable part is that we never explicitly tell the network what to detect. We do not program: ***"Filter #17 should detect eyes".*** Instead, through training on millions of images, the network automatically discovers useful patterns on its own.

So when we see: $224 \times 224 \times 64$, we can interpret it as: ***"The image has been analyzed by 64 different detectors, producing 64 different views of the same image".*** Each feature map highlights something different. One might emphasize edges. Another might emphasize textures. Another might emphasize circular shapes.

Together, they provide a rich description of the image that later layers can use to recognize increasingly complex objects.

In many ways, filters are the eyes of a Convolutional Neural Network. The more useful filters the network learns, the more detailed its understanding of the image becomes.

2. Leveraging the Pre-Trained VGG16 Model

Now that we have met VGG16, it is time to stop admiring the blueprint and actually use it. Remember the original motivation behind pre-trained models.

Training a deep convolutional neural network from scratch is expensive. It requires a large dataset, significant computing power, and a considerable amount of patience. VGG16 itself was trained on ImageNet, a dataset containing roughly 1.4 million images across 1,000 categories. That means the model has already spent a tremendous amount of effort learning how to recognize useful visual patterns.

So instead of starting from zero, we can simply borrow that knowledge. This idea is called **Feature Extraction**. The philosophy is surprisingly simple: ***Let***

VGG16 do what it is already good at—extracting visual features—and let us focus only on learning the final classification task.

For our cat-versus-dog problem, we do not need VGG16 to recognize 1,000 ImageNet categories. We only need it to answer a much simpler question: Is this image a cat or a dog? Because of that, we remove the original classifier and replace it with our own.

Step 1 — Load the Pre-Trained Convolutional Base

```
try:
    conv_base = keras.applications.vgg16.VGG16(
        weights="imagenet",
        include_top=False,
        input_shape=(180, 180, 3),)
    print("Loaded VGG16 pretrained on ImageNet.")

except Exception as e:
    print("Unable to download ImageNet weights.")
    print("Using weights=None instead.")
    print("Error:", repr(e))

    conv_base = keras.applications.vgg16.VGG16(
        weights=None,
        include_top=False,
        input_shape=(180, 180, 3),)
```

The most important parameter here is: `include_top=False` . The "***top***" refers to the original Dense classifier that predicts 1,000 ImageNet classes.

Since our task only contains two classes, those layers are no longer useful. We keep the convolutional base and discard the classifier. Think of it like hiring an experienced detective. We keep their investigative skills, but assign them a different case.

Step 2 — Freeze the Convolutional Base

```
conv_base.trainable = False
print("Trainable weights:", len(conv_base.trainable_weights))

conv_base.summary()
```

This is one of the most important steps in transfer learning. When we freeze a layer, its weights cannot be updated during training.

Why do we do this? Because the convolutional base has already learned valuable features from millions of images. If we immediately allow those weights to change, a small cat/dog dataset could easily destroy that knowledge.

Freezing the network protects the representations that VGG16 spent millions of images learning. It also dramatically reduces training time because only a small classifier needs to be optimized.

Step 3 — Build a New Classifier

```
inputs = keras.Input(shape=(32, 32, 3))

x = layers.Resizing(180, 180)(inputs)

x = layers.Lambda(lambda img:
                  keras.applications.vgg16.preprocess_input(img * 255.0),
                  name="vgg16_preprocess")(x)

x = conv_base(x, training=False)
x = layers.Flatten()(x)
x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.5)(x)

outputs = layers.Dense(1,activation="sigmoid")(x)

feature_model = keras.Model(inputs, outputs)
```

Notice what is happening. The image first passes through VGG16. VGG16 extracts hundreds of useful visual features. Those features are then flattened and fed into a small neural network classifier that learns how to distinguish cats from dogs.

In other words: ***Input Image*** → ***VGG16 Feature Extractor*** → ***New Dense Classifier*** → ***Cat or Dog***

The convolutional base acts like a feature-generating machine. The classifier acts like a decision maker.

Step 4 — Compile the Model

```
feature_model.compile(optimizer="rmsprop",
                      loss="binary_crossentropy", metrics=
["accuracy"],)
feature_model.summary()
```

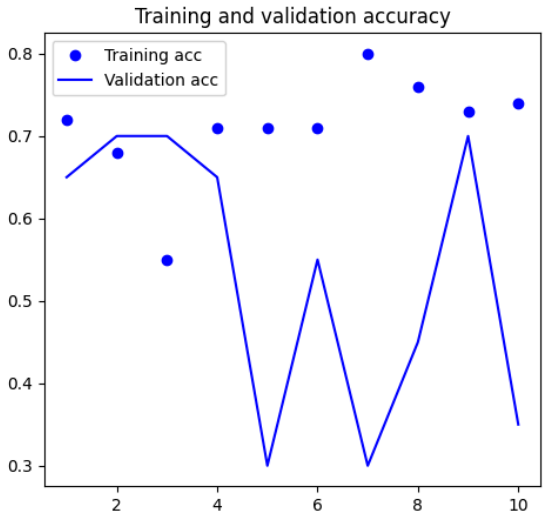
Since this is a binary classification problem, we use: `binary_crossentropy` and `sigmoid` for the output layer.

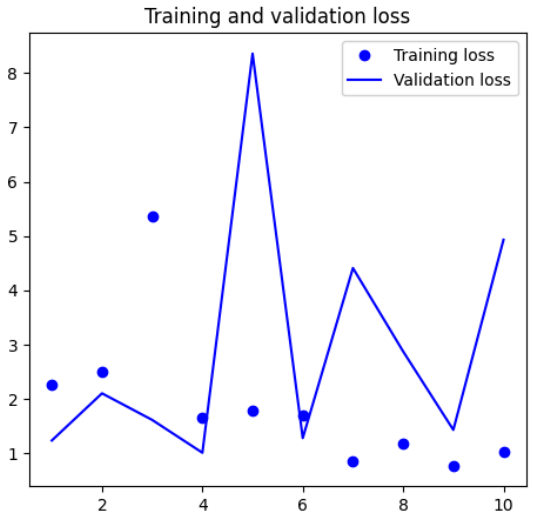
Step 5 — Train the New Classifier

```
RUN_FEATURE_EXTRACTION = True
FAST_RUN = True

if RUN_FEATURE_EXTRACTION:

    history_feature = feature_model.fit(
        x_cd_train,
        y_cd_train,
        epochs=2 if FAST_RUN else 10,
        batch_size=32,
        validation_data=(x_cd_val, y_cd_val),)

    plot_history(history_feature, "Feature Extraction with
VGG16")

else:
    print("Feature extraction training skipped.")
```

Assume you already know how to define the **plot_history()** function
Set RUN_FINE_TUNING = False and FAST_RUN = True to train 10 epochs

At this stage, only the Dense layers are learning. The convolutional base remains completely unchanged. This is Feature Extraction. The extracted features are fixed. The classifier adapts to the new problem.





3. Fine-Tuning the Pre-Trained Model

Feature extraction is powerful. But eventually a question appears: *What if the features learned from ImageNet are close to what we need, but not quite perfect?*

That is where Fine-Tuning comes in. Instead of treating the convolutional base as a frozen black box, we carefully allow part of it to continue learning.

The keyword here is "**carefully**". We do not want to retrain the entire VGG16 network. That would require significant computation and would dramatically increase the risk of overfitting. Instead, we only adjust the highest layers.

Why Only the Top Layers?

Remember how CNNs learn. Early layers learn simple patterns: Edges; Corners; Textures; Gradients. These features are useful for almost every computer vision problem.

Higher layers learn more specialized concepts: Faces; Fur; Wheels; Animal shapes. These features depend more heavily on the original training dataset. Therefore, if adaptation is needed, it is usually the higher layers that benefit most from fine-tuning.

For VGG16, we often choose Block 5.

Step 1 — Unfreeze Block 5

```
conv_base.trainable = True

for layer in conv_base.layers:
    layer.trainable = (layer.name.startswith("block5"))
```

Only layers belonging to Block 5 remain trainable. Everything else stays frozen.

Let's verify:

```
print("Trainable Layers:")
for layer in conv_base.layers:
    if layer.trainable:
        print("-", layer.name)
```

Trainable Layers:

- block5_conv1
- block5_conv2
- block5_conv3
- block5_pool

Step 2 — Recompile with a Small Learning Rate

```
feature_model.compile(optimizer=keras.optimizers.RMSprop(le
arning_rate=1e-5),
                      loss="binary_crossentropy",
                      metrics=["accuracy"],)
```

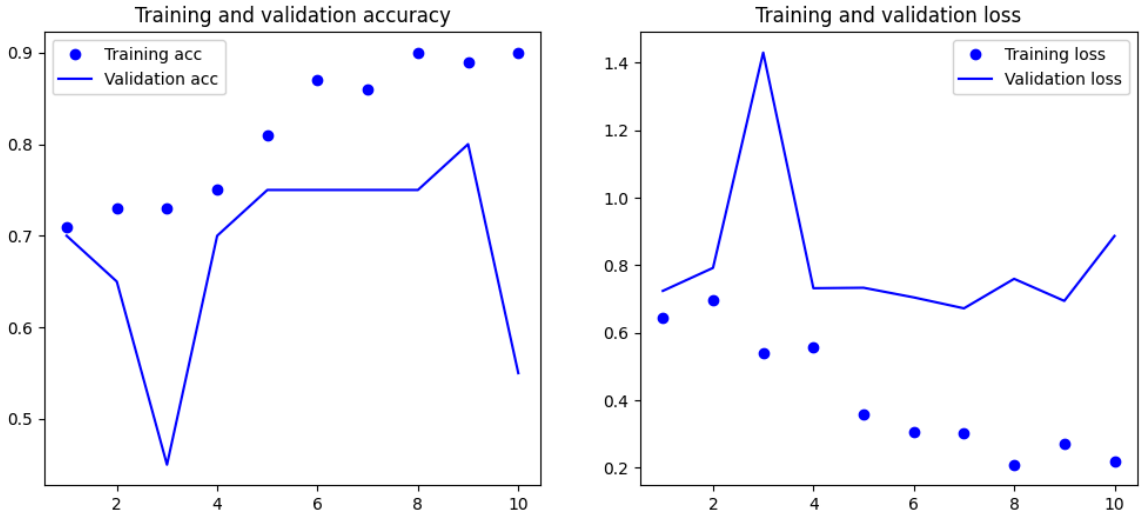
The learning rate becomes extremely important here. Why? Because VGG16 already contains valuable knowledge. A large learning rate could overwrite that knowledge almost immediately.

Fine-tuning should behave more like careful editing than complete rewriting. We make small adjustments rather than drastic changes.

Step 3 — Continue Training

```
RUN_FINE_TUNING = True
FAST_RUN = True

if RUN_FINE_TUNING:
    history_finetune = feature_model.fit(x_cd_train, y_cd_t
rain,
                                         epochs=2 if FAST_
RUN else 10,
                                         batch_size=32,
                                         validation_data=
(x_cd_val, y_cd_val))

    plot_history(history_finetune, "Fine-Tuning VGG16")
else:
    print("Fine-tuning skipped.")
```

Assume you already know how to define the **plot_history()** function
Set RUN_FINE_TUNING = False and FAST_RUN = True to train 10 epochs



Now two things are learning simultaneously: The new classifier and the layers inside Block 5. This allows VGG16 to slightly adjust its higher-level understanding to better fit our dataset.

- Think of feature extraction as hiring an experienced employee and letting them work exactly as they always have.

- Think of fine-tuning as giving that employee a short training course so they can adapt to your company's specific workflow.

4. Applying VGG16 to the Oxford-IIIT Pets Dataset

Up until this point, our cat-versus-dog example has been a useful demonstration of transfer learning. It showed us how a pre-trained model can be reused and adapted to a new classification task.

But let's be honest. A binary classification problem is relatively simple. The real strength of pre-trained CNNs becomes much more apparent when we move to a dataset containing many classes, subtle visual differences, and significantly greater variation.

This is where the **Oxford-IIIT Pets Dataset** comes in. The dataset contains thousands of images belonging to dozens of cat and dog breeds. Unlike the previous example, the challenge is no longer simply determining whether an animal is a cat or a dog.

Now the model must answer much more specific questions:

- Is this a Persian cat or a Siamese cat?
- Is this a Beagle or a Basset Hound?
- Is this a Maine Coon or a Norwegian Forest Cat?

To a human, these distinctions may already be difficult. To a machine, they are even harder because many breeds share similar colors, shapes, and textures. This is precisely the kind of problem where transfer learning shines.

Rather than forcing a network to learn everything from scratch, we leverage the knowledge already embedded inside VGG16. The model has previously seen millions of images and has already learned how to recognize edges, fur textures, facial structures, body shapes, and countless other visual patterns.

Our goal is not to teach VGG16 vision. Our goal is simply to teach it the specific breed categories that exist in this dataset. The workflow remains familiar:

1. Load the Oxford-IIIT Pets Dataset *(Here we only use the first 1000 samples for demonstration).*
2. Preprocess and resize images.
3. Load the pre-trained VGG16 convolutional base.

4. Replace the original ImageNet classifier.
5. Train a new classifier for pet breed recognition.
6. Optionally fine-tune the upper layers of VGG16.

The difference is that the dataset is now more complex, allowing us to better appreciate the power of transfer learning.

```
import tensorflow_datasets as tfds

# Load only the first 1000 samples of the train split
dataset, info = tfds.load("oxford_iiit_pet", split="train[:
1000]",
                          with_info=True, as_supervised=Tru
e)

# Verify the number of loaded samples
print(f"Loaded samples: {len(dataset)}")
print(info)
```

```
# Image preprocessing
IMG_SIZE = (180, 180)

def preprocess(image, label):
    image = tf.image.resize(image, IMG_SIZE)
    image = tf.cast(image, tf.float32) / 255.0

    return image, label
```

```
# Load VGG16 convolutional base

conv_base = keras.applications.VGG16(weights="imagenet",
                                     include_top=False, inp
ut_shape=(180, 180, 3))
conv_base.trainable = False
```

```
# Build classifier

inputs = keras.Input(shape=(180, 180, 3))

x = keras.applications.vgg16.preprocess_input(inputs * 255.
0)
x = conv_base(x, training=False)

x = layers.Flatten()(x)

x = layers.Dense(512, activation="relu")(x)
x = layers.Dropout(0.5)(x)

outputs = layers.Dense(37, activation="softmax")(x)

pet_model = keras.Model(inputs, outputs)
pet_model.compile(optimizer="adam", loss="sparse_categorica
l_crossentropy",
                  metrics=["accuracy"])
```

```
# Fit the model for the dataset
history = pet_model.fit(dataset.map(preprocess).batch(32),
                        epochs=10)
```

Once trained, the model can recognize dozens of different pet breeds while benefiting from the visual knowledge already learned from ImageNet.

Without transfer learning, achieving similar performance would require considerably more data, training time, and computational resources.

This is one of the reasons pre-trained models became a cornerstone of modern computer vision.

Summary

This section serves as a preview of **pre-trained convolutional neural networks** and demonstrates why they became one of the most important developments in modern Deep Learning.

Training a large CNN entirely from scratch is often expensive, time-consuming, and data-hungry. Pre-trained models offer a practical alternative by allowing us to reuse knowledge that has already been learned from massive datasets such as ImageNet.

Through **Feature Extraction**, we treated VGG16 as a fixed feature generator and trained only a new classifier. Through **Fine-Tuning**, we allowed a small portion of the network to adapt itself to a new dataset while preserving most of its previously learned knowledge.

Most importantly, this chapter illustrates a key idea that appears repeatedly throughout Machine Learning: Intelligence is not always built from the ground up. Sometimes the smartest approach is to stand on the shoulders of models that have already learned millions of examples before us.

VGG16 is only one example. Modern computer vision contains many other powerful architectures such as ResNet, Inception, EfficientNet, Vision Transformers (ViT), and countless specialized variants. Understanding how to leverage pre-trained models is therefore not merely a technique—it is a fundamental skill in practical Deep Learning.

And while we have only scratched the surface here, the ideas introduced in this section form the foundation for many real-world image classification systems used today.

References

https://www.geeksforgeeks.org/computer-vision/vgg-16-cnn-model/

https://www.youtube.com/watch?v=12UvnLp-8qg

Next Section:

- 15. Neural Networks for Images